

Version 1: I added to the ThreadInfo struct to store the threads index, start row, and end row. This holds each thread's row range. I calculated even row ranges, filled threadInfoArray[t].startRow and .endRow, and allocated pthread_t threadHandles[]. I changed the thread function to take ThreadInfo* and run computeRange(startRow, endRow). I also changed oneGeneration to create a new thread for each range, join it, call swapGrids(), and sleep between generations. Creating and destroying threads each generation is inefficient but easier to implement.

Version 2: I removed the thread creation from oneGeneration and instead created persistent threads in main. I also modified threadFunc() to loop forever with a while true loop. I removed all pthread_join calls. Persistent threads eliminate thread creation overhead. V2 prepares the structure needed for proper barriers and thread coordination in V3.

Version 3: I added the variables workMutex, workCond, doneCond, threadsAtBarrier, workAvailable, and quitFlag. All workers wait on workCond. The main now sets workAvailable to true and each worker computes its row range, which increments threadsAtBarrier. The last worker calls swapGrids, sets workAvailable to false, and signals doneCond. oneGeneration also got changed to set up the generation, broadcast, and wait for doneCond instead of computing anything. Barrier ensures all workers finish a generation before swapping, persistent threads minimize overhead, and condition variables prevent busy-waiting

Version 4: I replaced threadFunc with a random updater, which picks random i and j values using rand_r, runs cellNewState with i and j, and writes directly into the current grid. It loops until the quitFlag is set to true. I also commented out oneGeneration since v4 ignores generation logic. Threads can freely overwrite shared data.

EC2: to synchronize access to the entire grid, we would use a 2D array of mutexes, one mutex per cell. Before a thread updates cell (i,j), it would need to lock not just that cell, but all its neighboring 3x3 cells, since cellNewState(i, j) reads the neighbors as well. The thread could compute the indices of the 3x3 neighborhood around (i, j), acquire all 9 mutexes to avoid deadlock, recompute the neighbor count and new state, update currentGrid[i][j] in place, and then release all 9 locks. This resembles the dining philosophers style synchronization problem.

EC1: I replaced the C-style rand() with std::mt19937 and std::bernoulli_distribution to get higher-quality, more controllable randomness and to follow modern C++ best practices.

EC5: I added a new predefined rule, HighLife, B36/S23